Proyecto final de Lenguajes Formales

FIRST/FOLLOW, analizadores LL(1) y SLR(1) y evaluación de un SDD

Roger Steward Arenas Rodríguez Harol Hernán Cuéllar Narváez
Luis Fernando Cardona

13 de noviembre de 2026

Introducción

En este proyecto programamos en Java 21 (con Spring Boot) lo que vimos en las secciones 4.4, 4.5, 4.6, 5.1 y 5.2 del libro de Aho et al., *Compilers: Principles, Techniques, and Tools* (2.ª ed.):

- el cálculo de los conjuntos FIRST y FOLLOW;
- un analizador LL(1) con tabla y uno SLR(1);
- un evaluador de SDD que arma el grafo de dependencias y saca un orden topológico.

En cada parte contamos qué algoritmo del libro seguimos y cómo guardamos los datos. Al final mostramos un ejemplo con una gramática que inventamos, el SDD que diseñamos y los problemas que nos fuimos encontrando.

1. Cómo está organizado el proyecto

El programa principal funciona por consola. No usamos generadores de analizadores ni librerías de análisis sintáctico, todo está hecho a mano con la biblioteca estándar de Java. El código está en `src/main/java/co/edu/eafit/lf` y lo repartimos en estos paquetes:

Paquete	Qué tiene
`grammar`	`Production` (cabeza y cuerpo), `Grammar` (la gramática) y `GrammarReader` (lee el formato de la sección 2.2 del enunciado).
`analysis`	`FirstFollow`: los conjuntos FIRST y FOLLOW.
`parsing`	`LL1Parser`, `SLR1Parser`, la interfaz `Parser` que tienen en común, el árbol de derivación (`ParseNode`) y la traza paso a paso (`Trace`).
`sdd`	`Sdd` (el SDD guardado como datos), `BaseNumberSdd` (nuestro SDD) y `SddEvaluator` (el evaluador).
`cli`	`ConsoleRunner`: la entrada y salida por consola.
`automata`	Lo de las entregas anteriores: `SubsetConstruction` (AFN a AFD), `DfaSimulator` y `DfaMinimizer`.
`web`	`ApiController` y `AutomataController`: la API que usa la página web.

La página web. Además de la consola hicimos una página para ver todo de forma más gráfica. Si se corre el programa con `--web`, arranca un servidor en `http://localhost:8080`. La página (`src/main/resources/static`) le manda cada petición a los controladores del paquete `web`, y estos llaman a las mismas clases que usa la consola. O sea, las tablas, las trazas y la evaluación del

SDD que se ven en la página las calcula el mismo código Java. De paso le metimos las entregas anteriores del curso (AFN a AFD, simulación y minimización de AFD). La página en sí no calcula nada: solo le manda los datos a Java y dibuja lo que le responde.

Convenciones. Como dice el enunciado, cada símbolo es un carácter (`char`). Los no terminales son letras mayúsculas, la `e` es ε y `$` es el fin de cadena, que el programa le pega a cada cadena. Una producción $A \to \varepsilon$ la guardamos con el cuerpo vacío; así nunca se confunde ε con un terminal.

Estructuras de datos.

- Gramática: una lista de producciones, un mapa `Map<Character, List<Production>>` con las producciones de cada cabeza y dos `TreeSet` (no terminales y terminales), para que todo salga ordenado.
- FIRST y FOLLOW: `Map<Character, Set<Character>>`. La ε se guarda como el carácter `e`.
- Tabla LL(1): `Map<Character, Map<Character, Production>>`, que es justo $M[A, a]$.
- Autómata LR(0): una lista de estados `List<Set<Item>>`. Cada `Item` es un *record* con la producción y la posición del punto. Con un `HashMap<Set<Item>, Integer>` sabemos si un estado ya existía, y las transiciones van en `List<Map<Character, Integer>>`.
- Tablas SLR(1): `List<Map<Character, Action>>` para ACTION (desplazar, reducir o aceptar), y las transiciones con no terminales para GOTO.
- Pilas de los analizadores: `ArrayDeque`.
- Árbol de derivación: `ParseNode`, que guarda el símbolo, la producción usada, los hijos y un mapa con los atributos.
- Grafo de dependencias: `Map<Instance, List<Instance>>`, que dice de qué atributos depende cada uno. Además se guardan los grados de entrada y los sucesores para el orden topológico.

Compilar y correr.

- `mvnw clean package` compila y corre las 43 pruebas unitarias.
- El programa se corre con `java -jar target/lenguajes-formales.jar` y lee la gramática de la entrada estándar.
- Opciones: `--sets` (imprime FIRST y FOLLOW), `--tables` (imprime las tablas y trazas que usamos en este informe), `--sdd` y `--sdd --circular` (Item 3), y `--web` (la página).

2. Item 1: FIRST y FOLLOW

Aquí seguimos la sección 4.4.2. Los dos conjuntos se calculan por *punto fijo*: aplicamos las reglas del libro a todas las producciones una y otra vez, hasta que una vuelta completa no cambia ningún conjunto. Eso siempre termina, porque los conjuntos solo crecen.

- $\text{FIRST}(a) = \{a\}$ si a es terminal. Para una producción $X \to Y_1 \cdots Y_k$ metemos $\text{FIRST}(Y_1) \setminus \{\varepsilon\}$; si Y_1 puede volverse ε, seguimos con Y_2, y así. Si todos pueden volverse ε (o el cuerpo es ε), metemos ε.
- Con la misma idea sacamos $\text{FIRST}(X_1 X_2 \cdots X_n)$ de una cadena entera (método `firstOf`), que se usa para la tabla LL(1) y para FOLLOW.

- FOLLOW: primero ponemos `$` en FOLLOW(S). Luego, para cada $A \to \alpha B \beta$, metemos FIRST(β) $\setminus \{\varepsilon\}$ en FOLLOW(B). Si β puede volverse ε (o no hay β), también metemos todo FOLLOW(A) en FOLLOW(B).

Con `--sets` se imprime una línea por no terminal, en orden alfabético, primero los FIRST y luego los FOLLOW. Dentro de las llaves van los terminales en orden ASCII, después la `e` y de último el `$`. Aquí hubo un detalle: en ASCII el `$` va antes que las letras, así que no bastaba con ordenar el conjunto. Por eso el código separa la ε y el `$` y los pone al final.

3. Item 2: analizadores LL(1) y SLR(1)

3.1. LL(1)

- **¿Es LL(1)?** (comienzo de la sección 4.4.3). Para cada par de alternativas $A \to \alpha \mid \beta$ revisamos tres cosas:

  1. que α y β no puedan empezar con el mismo terminal;
  2. que a lo sumo una de las dos pueda volverse ε;
  3. que si β puede volverse ε, entonces α no pueda empezar con algo de FOLLOW(A) (y al revés).

  La gramática se usa tal como llega: no le quitamos la recursión por la izquierda ni nada.

- **La tabla** (Algoritmo 4.31). Para cada $A \to \alpha$ ponemos esa producción en $M[A, a]$ para cada a de FIRST(α) (sin la ε). Si ε está en FIRST(α), también la ponemos en $M[A, b]$ para cada b de FOLLOW(A). Si una casilla termina con dos producciones, hay conflicto.

- **El análisis** (Algoritmo 4.34, Fig. 4.20). Es el analizador predictivo con pila, que arranca con `S$`:

  - si arriba hay un terminal igual al de la entrada, se emparejan y se avanza;
  - si arriba hay un no terminal X, se cambia por el cuerpo de $M[X, a]$;
  - si la casilla está vacía, es error.

  Cada vez que se expande una X se le crean los hijos en el árbol de derivación.

3.2. SLR(1)

- **Gramática aumentada e ítems LR(0)** (sección 4.6.2). Le agregamos $S' \to S$. Para la cabeza S' usamos un carácter reservado que nunca sale en la entrada.

- **CLOSURE y GOTO**. CLOSURE (Fig. 4.32) se calcula con una cola de trabajo. GOTO(I, X) es la clausura de los ítems de I después de pasar el punto por encima de X.

- **Colección canónica** (Fig. 4.33). Hacemos un recorrido en anchura desde CLOSURE($\{[S' \to \cdot S]\}$). Como los símbolos se recorren en orden, los estados salen siempre con el mismo número.

- **Tablas ACTION y GOTO** (Algoritmo 4.46):

  - si $[A \to \alpha \cdot a\beta]$ está en I_i, se desplaza;
  - si $[A \to \alpha\cdot]$ está en I_i y $A \neq S'$, se reduce por $A \to \alpha$ en cada a de FOLLOW(A);
  - si $[S' \to S\cdot]$ está en I_i, se acepta en `$`.

  La gramática es SLR(1) solo si ninguna casilla queda con dos acciones distintas, o sea, si no hay conflictos desplazar/reducir ni reducir/reducir (sección 4.5.4).

- **El análisis** (Algoritmo 4.44, Fig. 4.36). Usa una pila de estados y otra de nodos que se mueven juntas. Cuando se reduce por $A \to \beta$, se sacan $|\beta|$ nodos y quedan como hijos del nodo nuevo A.

3.3. Qué hace el programa

Después de leer la gramática se arman los dos analizadores, y según el resultado pasa una de estas cosas (casos 1 a 4 de la sección 2.2.2):

- si es LL(1) y SLR(1), muestra el menú `Select a parser (T: for LL(1), B: for SLR(1), Q: quit):`;
- si solo es una de las dos, imprime `Grammar is LL(1).` o `Grammar is SLR(1).`;
- si no es ninguna, imprime `Grammar is neither LL(1) nor SLR(1).`

Luego lee cadenas hasta una línea vacía y responde `yes` o `no` a cada una. Los tres ejemplos del enunciado están en las pruebas automáticas.

4. Ejemplo con una gramática nuestra

Inventamos una gramática para listas anidadas, como `[n,[n,n],[]]`, donde `n` es un elemento:

$$S \to [\,L\,] \mid n \qquad L \to S\,R \mid \varepsilon \qquad R \to ,\, S\,R \mid \varepsilon$$

Así se escribe en el formato de entrada:

```
3
S -> [L] n
L -> SR e
R -> ,SR e
```

Todo lo de esta sección lo sacamos del mismo programa con `--tables`.

4.1. FIRST y FOLLOW

	FIRST	FOLLOW
L	{`[`, `n`, ε}	{`]`}
R	{`,`, ε}	{`]`}
S	{`[`, `n`}	{`,`, `]`, `$`}

4.2. Tabla LL(1)

Ninguna casilla tiene dos producciones, así que la gramática es LL(1).

	`,`	`[`	`]`	`n`	`$`
L		$L \to SR$	$L \to \varepsilon$	$L \to SR$	
R	$R \to ,SR$		$R \to \varepsilon$		
S		$S \to [L]$		$S \to n$	

4.3. Autómata LR(0) y tabla SLR(1)

I_0	$S' \to \cdot S,\ S \to \cdot[L],\ S \to \cdot n$	I_6	$S \to [L]\cdot$
I_1	$S' \to S\cdot$	I_7	$R \to, \cdot SR,\ S \to \cdot[L],\ S \to \cdot n$
I_2	$S \to [\cdot L],\ L \to \cdot SR,\ L \to \cdot,\ S \to \cdot[L],\ S \to \cdot n$	I_8	$L \to SR\cdot$
I_3	$S \to n\cdot$	I_9	$R \to, S \cdot R,\ R \to \cdot, SR,\ R \to \cdot$
I_4	$S \to [L\cdot]$	I_{10}	$R \to, SR\cdot$
I_5	$L \to S \cdot R,\ R \to \cdot, SR,\ R \to \cdot$		

Transiciones: $I_0 \xrightarrow{S} I_1$, $I_0 \xrightarrow{[} I_2$, $I_0 \xrightarrow{n} I_3$, $I_2 \xrightarrow{L} I_4$, $I_2 \xrightarrow{S} I_5$, $I_2 \xrightarrow{[} I_2$, $I_2 \xrightarrow{n} I_3$, $I_4 \xrightarrow{]} I_6$, $I_5 \xrightarrow{,} I_7$, $I_5 \xrightarrow{R} I_8$, $I_7 \xrightarrow{S} I_9$, $I_7 \xrightarrow{[} I_2$, $I_7 \xrightarrow{n} I_3$, $I_9 \xrightarrow{,} I_7$, $I_9 \xrightarrow{R} I_{10}$.

	ACTION					GOTO		
Estado	,	[	]	n	$	L	R	S
0		s2		s3				1
1					acc			
2		s2	r($L \to \varepsilon$)	s3		4		5
3	r($S \to n$)		r($S \to n$)		r($S \to n$)			
4			s6					
5	s7		r($R \to \varepsilon$)				8	
6	r($S \to [L]$)		r($S \to [L]$)		r($S \to [L]$)			
7		s2		s3				9
8			r($L \to SR$)					
9	s7		r($R \to \varepsilon$)				10	
10			r($R \to, SR$)					

Ninguna casilla tiene dos acciones, así que también es SLR(1).

4.4. Paso a paso con la cadena `[n,[]]`

Con el LL(1) (las columnas son como las de la Fig. 4.21 y el tope de la pila está a la izquierda):

```
Matched | Stack  | Input   | Action
--------+--------+---------+----------------
        | S$     | [n,[]]$ | output S -> [L]
        | [L]$   | [n,[]]$ | match [
[       | L]$    | n,[]]$  | output L -> SR
[       | SR]$   | n,[]]$  | output S -> n
[       | nR]$   | n,[]]$  | match n
[n      | R]$    | ,[]]$   | output R -> ,SR
[n      | ,SR]$  | ,[]]$   | match ,
[n,     | SR]$   | []]$    | output S -> [L]
[n,     | [L]R]$ | []]$    | match [
[n,[    | L]R]$  | ]]$     | output L -> e
[n,[    | ]R]$   | ]]$     | match ]
[n,[]   | R]$    | ]$      | output R -> e
[n,[]   | ]$     | ]$      | match ]
[n,[]]  | $      | $       | accept
```

Con el SLR(1) (columnas como las de la Fig. 4.38):

```
Stack         | Symbols | Input   | Action
--------------+---------+---------+-------------------
0             |         | [n,[]]$ | shift 2
0 2           | [       | n,[]]$  | shift 3
0 2 3         | [n      | ,[]]$   | reduce by S -> n
```

```
0 2 5         | [S      | ,[]]$   | shift 7
0 2 5 7       | [S,     | []]$    | shift 2
0 2 5 7 2     | [S,[    | ]]$     | reduce by L -> e
0 2 5 7 2 4   | [S,[L   | ]]$     | shift 6
0 2 5 7 2 4 6 | [S,[L]  | ]$      | reduce by S -> [L]
0 2 5 7 9     | [S,S    | ]$      | reduce by R -> e
0 2 5 7 9 10  | [S,SR   | ]$      | reduce by R -> ,SR
0 2 5 8       | [SR     | ]$      | reduce by L -> SR
0 2 4         | [L      | ]$      | shift 6
0 2 4 6       | [L]     | $       | reduce by S -> [L]
0 1           | S       | $       | accept
```

Se ve bien la diferencia entre los dos: el LL(1) arranca usando $S \to [L]$ y el SLR(1) termina reduciendo por $S \to [L]$. Arman el mismo árbol, uno de arriba hacia abajo y el otro de abajo hacia arriba.

5. Item 3: nuestro SDD

5.1. Qué calcula

Nuestro SDD saca el valor decimal de un número escrito en cualquier base, con la forma `base#dígitos`. Por ejemplo, `8#17` es 17 en base 8, o sea $1 \cdot 8 + 7 = 15$. La gramática es

$$S \to D \# N \qquad N \to D\,R \qquad R \to D\,R \mid \varepsilon \qquad D \to 0 \mid 1 \mid \cdots \mid 9$$

Tiene 4 no terminales y 14 producciones, R es recursivo y es LL(1) y SLR(1) (lo revisamos con el mismo programa), así que las cadenas las analizamos con nuestro LL(1). Cumple G1, G2 y G3: no sale en el libro ni en el enunciado. La idea de ir acumulando el valor con un atributo heredado se parece a los ejemplos de números del libro, pero lo de heredar la base y el acumulador lo pensamos nosotros.

5.2. Atributos

Símbolo	Atributo	Tipo	Valor	Para qué sirve
D	`val`	sintetizado	entero	valor del dígito
N	`base`	heredado	entero	base en la que se lee el número
N	`val`	sintetizado	entero	valor del número
R	`base`	heredado	entero	base del número
R	`acc`	heredado	entero	lo que valen los dígitos a la izquierda de R
R	`val`	sintetizado	entero	valor final del número
S	`val`	sintetizado	entero	el resultado (atributo principal)
terminales	`lexval`	sintetizado	carácter	el mismo carácter (lo da el escáner)

5.3. Reglas semánticas

Producción	Reglas semánticas
$S \to D \# N$	$N.base = D.val$; $S.val = N.val$
$N \to D\,R$	$R.base = N.base$; $R.acc = D.val$; $N.val = R.val$
$R \to D\,R_1$	$R_1.base = R.base$; $R_1.acc = R.acc \cdot R.base + D.val$; $R.val = R_1.val$
$R \to \varepsilon$	$R.val = R.acc$
$D \to k \;\; (k = 0, \ldots, 9)$	$D.val =$ valor numérico de $k.lexval$

En palabras: la base baja por el árbol, y `acc` va armando el número de izquierda a derecha, igual que como se hace a mano (lo que llevo, por la base, más el dígito nuevo). Cuando ya no quedan dígitos ($R \to \varepsilon$), lo acumulado es el resultado y sube hasta S.

¿Cumple lo de la sección 5.1.1? Sí. Cada regla define un atributo sintetizado de la cabeza o un heredado de un símbolo del cuerpo, y solo usa atributos de la misma producción. Además, en cada producción cada uno de esos atributos se define una sola vez:

- en $S \to D\#N$: $S.val$ y $N.base$ (D no tiene heredados);
- en $N \to DR$: $N.val$, $R.base$ y $R.acc$;
- en $R \to DR_1$: $R.val$, $R_1.base$ y $R_1.acc$;
- en $R \to \varepsilon$: $R.val$;
- en $D \to k$: $D.val$.

El símbolo inicial S no tiene heredados, y el SDD usa tanto atributos sintetizados como heredados.

5.4. Ejemplo a mano: `8#17`

En la Figura 1 está el árbol anotado. Los nodos van numerados en preorden, igual que en lo que imprime el programa.

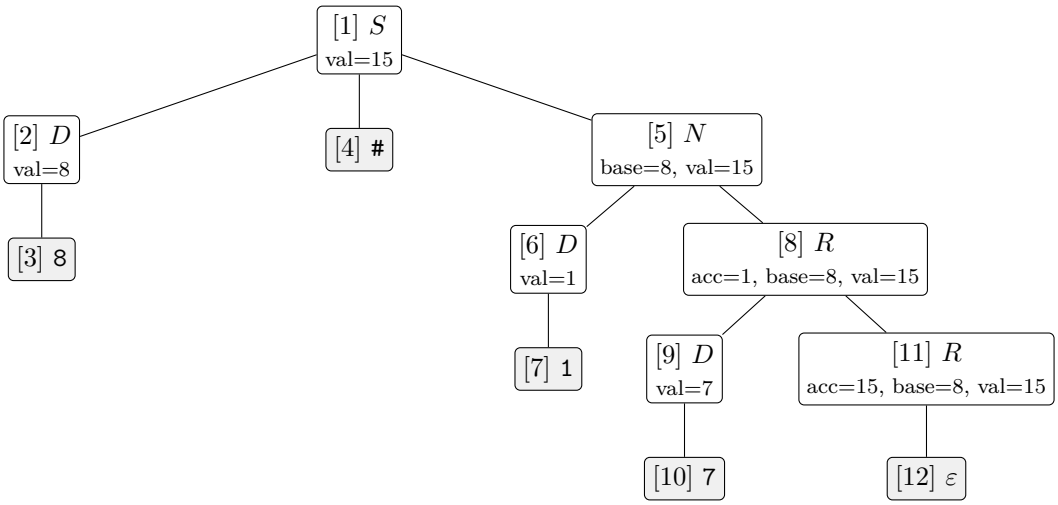


Figura 1: Árbol de derivación anotado de `8#17`.

Grafo de dependencias. Como en la sección 5.2.1, ponemos un vértice por cada atributo de cada nodo (también los `lexval` de las hojas) y una flecha desde cada atributo que usa una regla hasta el atributo que esa regla calcula (Figura 2). El número de cada vértice es su posición en el orden topológico que damos después.

Orden de evaluación. Este es un orden topológico válido (sección 5.2.2) y es justo el que saca el programa (entre paréntesis va el número del nodo):

1	`8`.lexval (3)	5	D.val (2)	9	R.acc (8)	13	R_1.val (11)
2	`#`.lexval (4)	6	D.val (6)	10	R.base (8)	14	R.val (8)
3	`1`.lexval (7)	7	D.val (9)	11	R_1.base (11)	15	N.val (5)
4	`7`.lexval (10)	8	N.base (5)	12	R_1.acc (11)	16	S.val (1)

Siguiendo ese orden: los $D.val$ dan 8, 1 y 7; $N.base = 8$; $R.acc = 1$; $R.base = 8$; $R_1.base = 8$; $R_1.acc = 1 \cdot 8 + 7 = 15$; $R_1.val = 15$; $R.val = 15$; $N.val = 15$ y al final $S.val = 15$.

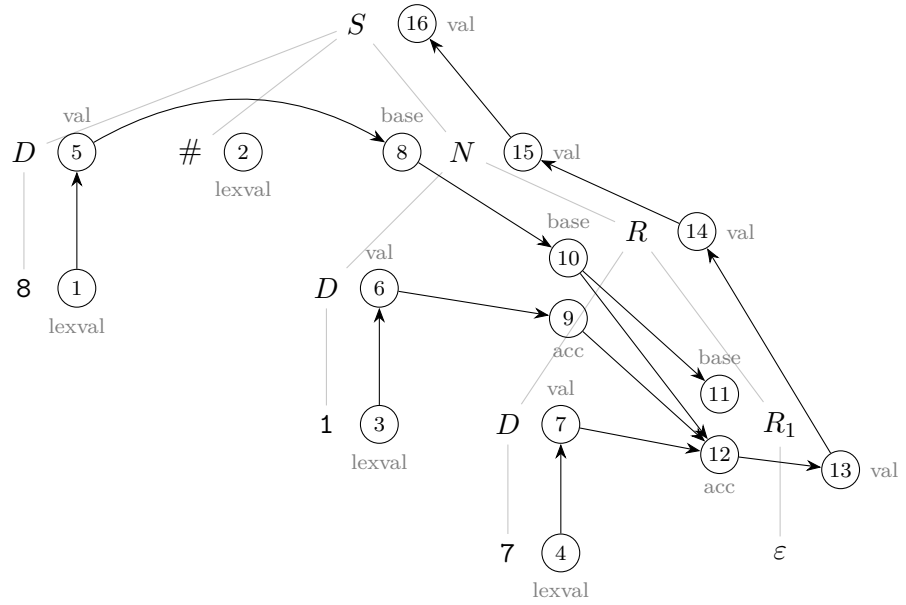


Figura 2: Grafo de dependencias de `8#17`. El árbol está en gris y cada círculo es un atributo.

5.5. ¿S-atribuido o L-atribuido?

No es S-atribuido (sección 5.2.3), porque tiene atributos heredados ($N.base$, $R.base$ y $R.acc$). **Sí es L-atribuido** (sección 5.2.4), porque cada heredado de un símbolo X_i del cuerpo solo depende de heredados de la cabeza o de atributos de símbolos que están a su izquierda:

- $N.base = D.val$ en $S \to D\#N$: la D está a la izquierda de N.
- $R.base = N.base$ y $R.acc = D.val$ en $N \to DR$: uno viene de la cabeza y el otro de un símbolo a la izquierda.
- $R_1.base = R.base$ y $R_1.acc = R.acc \cdot R.base + D.val$ en $R \to DR_1$: vienen de la cabeza y de D, que está a la izquierda de R_1.

Eso quiere decir que se podría evaluar en un solo recorrido de izquierda a derecha y que el grafo nunca va a tener ciclos. Pero el enunciado pide el método general con el grafo de dependencias, así que eso fue lo que hicimos.

5.6. Cómo lo programamos

1. **El SDD es datos.** En `Sdd` guardamos la lista de reglas de cada producción. Cada regla dice qué atributo calcula, qué atributos usa (posición 0 para la cabeza y k para el k-ésimo símbolo del cuerpo) y la función que hace el cálculo. `BaseNumberSdd` solo tiene esos datos; el evaluador no sabe nada de bases ni de números, así que serviría para otro SDD.

2. **El árbol.** La cadena se analiza con nuestro LL(1), que arma el árbol. Cada nodo guarda su símbolo, la producción que se usó, sus hijos y sus atributos.

3. **El grafo.** Numeramos los nodos en preorden. En cada nodo interior tomamos las reglas de su producción y agregamos una flecha de cada atributo usado al atributo calculado; las hojas aportan su `lexval`. También revisamos si un atributo se define dos veces o si se usa uno que nadie define.

4. **El orden y la evaluación.** El orden topológico sale con el algoritmo de Kahn: se van sacando los vértices que ya no esperan a nadie y se les quitan sus flechas. Si al final quedan vértices sin sacar, es porque hay un ciclo, y el programa avisa que el SDD es circular y para. Si no, calcula los atributos en ese orden.

5. **La salida.** Si la cadena no es del lenguaje imprime `no`. Si sí lo es, imprime el árbol anotado (un nodo por línea, con sangría según la profundidad), el orden de evaluación y el valor de $S.val$.

5.7. Pruebas

Cadena	Qué probamos	Resultado
`2#0`	la cadena más corta	$S.val = 0$
`8#17`	el ejemplo a mano	$S.val = 15$
`2#1011`	un binario	$S.val = 11$
`2#11111111`	recursión más profunda (8 niveles de R)	$S.val = 255$
`9#`	no es del lenguaje	`no`
`#12`	no es del lenguaje	`no`

Por ejemplo, para `2#0` el programa imprime todo esto:

```
Annotated parse tree:
[1] S  val=0
  [2] D  val=2
    [3] 2  lexval=2
  [4] #  lexval=#
  [5] N  base=2, val=0
    [6] D  val=0
      [7] 0  lexval=0
    [8] R  acc=0, base=2, val=0
      [9] e
Evaluation order:
1. 2.lexval (node 3)
2. #.lexval (node 4)
3. 0.lexval (node 7)
4. D.val (node 2)
5. D.val (node 6)
6. N.base (node 5)
7. R.acc (node 8)
8. R.base (node 8)
9. R.val (node 8)
10. N.val (node 5)
11. S.val (node 1)
S.val = 0
```

¿Y si hay un ciclo? Para probarlo, con `--sdd --circular` cambiamos la regla $R.acc = D.val$ de $N \to DR$ por $R.acc = N.val$. Eso arma el ciclo $N.val \to R.acc \to R_1.acc \to R_1.val \to R.val \to N.val$, y el programa imprime:

```
The SDD is circular: the dependency graph has a cycle.
```

6. Problemas que tuvimos y cómo los resolvimos

Spring Boot ensuciaba la salida. Spring Boot imprime un banner y mensajes de registro, y eso dañaba el formato exacto que pide el enunciado (`yes`/`no`). Apagamos el banner y los registros cuando el programa corre por consola.

Un carácter invisible al principio. Cuando le pasábamos la entrada con una tubería de PowerShell (`Get-Content archivo | java ...`), la primera línea llegaba con un carácter

U+FEFF (BOM) pegado. Por eso el programa no podía leer el número de no terminales, y en el modo SDD rechazaba la primera cadena. Lo arreglamos leyendo en UTF-8 y saltando ese carácter si aparece al inicio.

Cómo guardar ε. En la entrada la `e` es ε, pero no la podíamos tratar como terminal. Por eso $A \to \varepsilon$ se guarda con el cuerpo vacío y la `e` solo aparece como marca dentro de los FIRST. Si alguien escribe la cadena `e`, se toma como la cadena vacía.

El orden al imprimir los conjuntos. El formato pide los terminales en orden ASCII, después la `e` y de último el `$`. Como el `$` va antes que las letras en ASCII, los separamos y los pusimos al final a mano.

El símbolo S'. No podía ser una mayúscula, porque cualquiera de ellas puede ser un no terminal de la gramática. Usamos un carácter reservado que nunca sale en la entrada y lo mostramos como `S'`.

Estados LR(0) repetidos y números que cambiaban. Para saber si un estado ya existía, comparamos conjuntos de ítems usando un `HashMap` con el conjunto como clave. Como los ítems son *records*, ya traen el `equals` y el `hashCode` por valor. Además recorremos los símbolos en orden, para que los estados salgan con el mismo número siempre y coincidan con los de este informe.

Las hojas ε. En el libro ε aparece como hoja del árbol. Hicimos que los dos analizadores creen una hoja `e` al usar $A \to \varepsilon$; así los dos dan el mismo árbol y el SDD puede usar cualquiera.

No repetir código para la traza. Para mostrar el paso a paso sin escribir dos veces cada analizador, cada uno tiene un solo ciclo que va anotando los pasos solo cuando se le pide la traza.

Algo que le falta al SDD. El SDD no revisa que cada dígito sea menor que la base: `2#19` da $1 \cdot 2 + 9 = 11$. Se podría arreglar con un atributo sintetizado que diga si el número es válido.

Referencias

[1] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman. *Compilers: Principles, Techniques, and Tools*, 2.ª ed. Addison-Wesley, 2006.